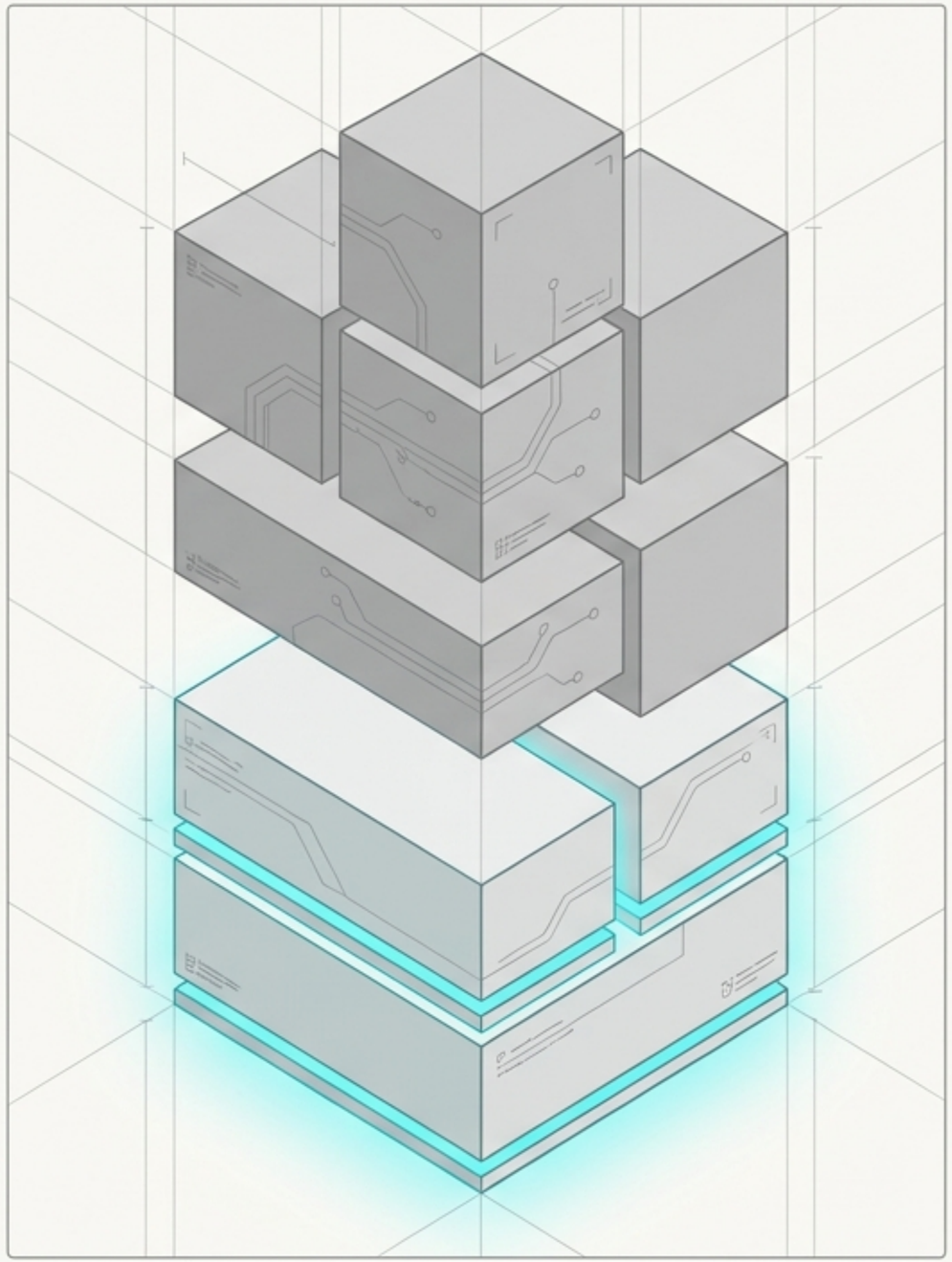


RUNTIME: CLAUDE OPUS 4.8 | FRAMEWORK: PRODUCTION CASE STUDY

Architecting Agents for Prompt Caching

How **Viktor** built a thread engine **around append-only logs** and in-cache compaction to cut LLM **costs by 80%.**

NotebookLM

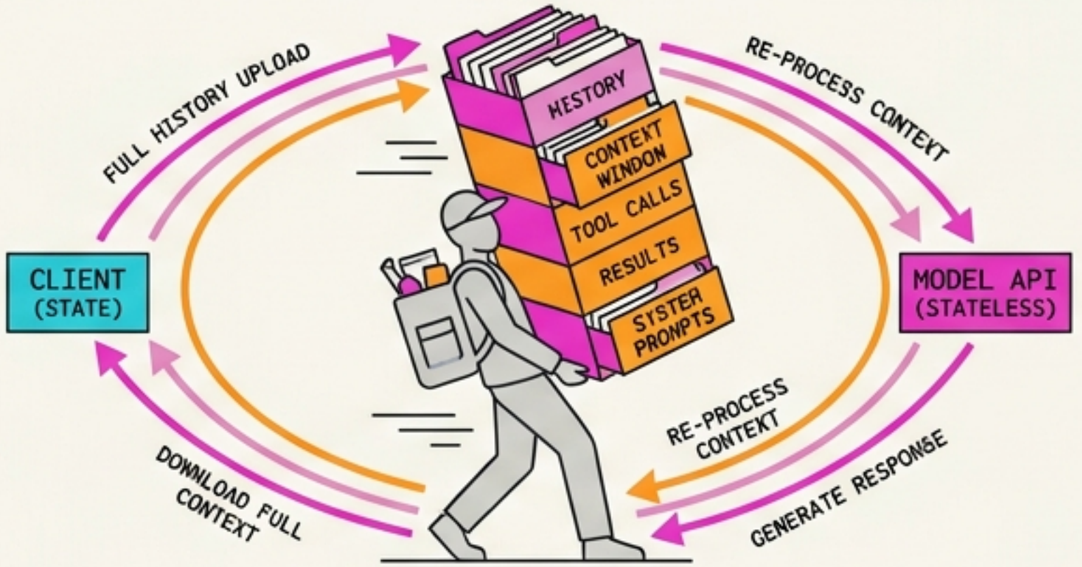
The Core Illusion of LLM Agents

THE EXPECTATION: An open channel where you only transmit new thoughts.



An open channel where you only transmit new thoughts.

THE REALITY: Stateless APIs. Every tool call, result, and historical message must be completely re-transmitted on every single turn.

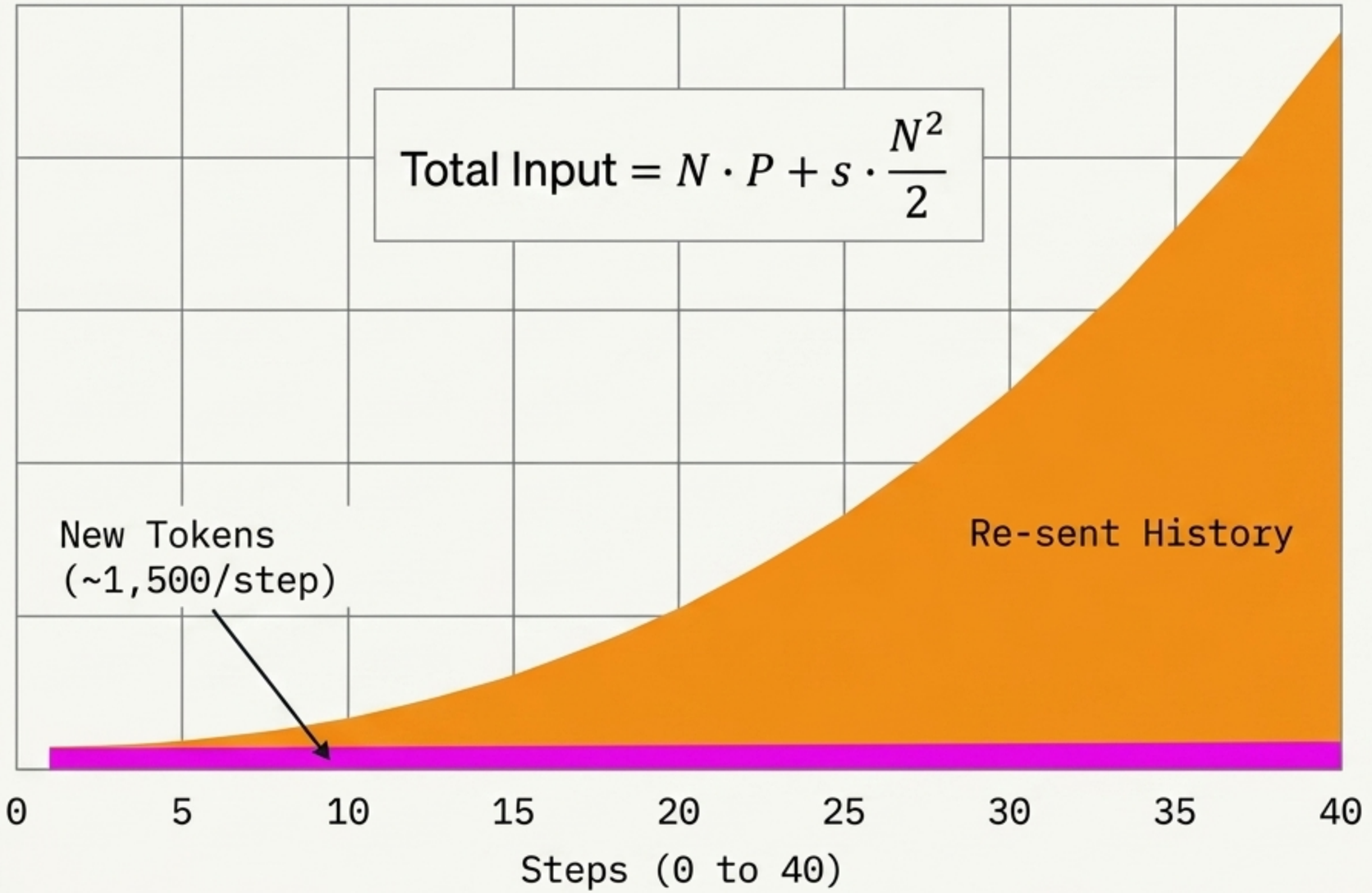


Stateless APIs. Every tool call, result, and historical message must be completely re-transmitted on every single turn.

An agent loop working at machine speed is just a model conversing with itself. Re-sending history on every step creates a devastating structural bottleneck.

NotebookLM

The Quadratic Cost of Memory



Final Transcript Size:
85,000 tokens

Total Tokens Transmitted:
2,170,000 input tokens

You are paying for the exact same early tokens up to 40 times.

Double the length of a task, and you pay four times as much for the tail.

NotebookLM

The Economics of the Cache

WITHOUT CACHING
INPUT: 2,170,000 tok @ $5.00/M = $10.85
OUTPUT: 20,000 tok @ $25.00/M = $0.50
TOTAL = $11.35

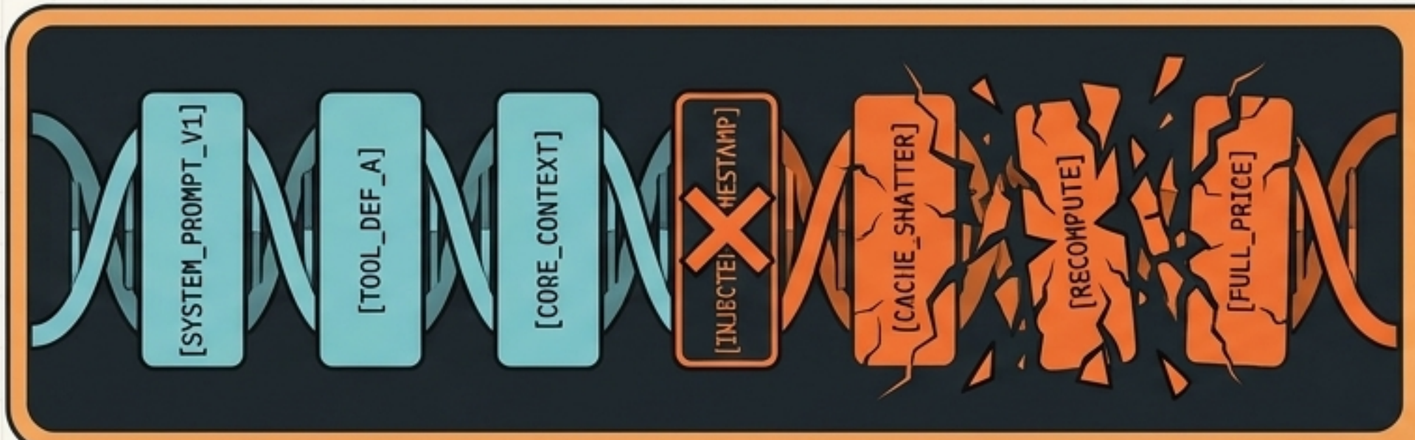
WITH PROMPT CACHING
CACHE WRITE: 83,500 tok @ $6.25/M = $0.52 (1.25x premium)
CACHE READ: 2,086,500 tok @ $0.50/M = $1.04 (0.1x discount)
OUTPUT: 20,000 tok @ $25.00/M = $0.50
TOTAL = $2.07



LATENCY BONUS: Cached tokens completely skip prefill compute. Time-to-first-token on an 80K context drops from seconds to near-zero.

NotebookLM

The Catch: Two Brutal Constraints



Constraint 1: The Byte-Stable Prefix

The exact prefix match rule. Change one character in a system prompt, reorder a tool, or inject a timestamp at position zero, and the entire cache shatters. You pay full price to recompute.



Constraint 2: The Impatient TTL

Anthropic's standard cache entry dies after exactly 5 minutes of silence. If you wait too long between agent actions, the prefix expires.

TAKEAWAY: Caching is not a billing toggle; it is a rigid architectural constraint. You must engineer your backend specifically to keep the prefix stable and survive cache death.

NotebookLM

Fix 1: The SDK Decision (Protecting Position Zero)

THE NAIVE WAY: Dynamic Tool Schemas

Context window



Injecting dynamic schemas means the toolset changes on every step.
Since tools sit at the front of the prompt, position zero changes, and everything downstream breaks.

THE VIKTOR WAY: Fixed Native Tools via SDK

Fixed Native Tools
(Bash, File I/O)

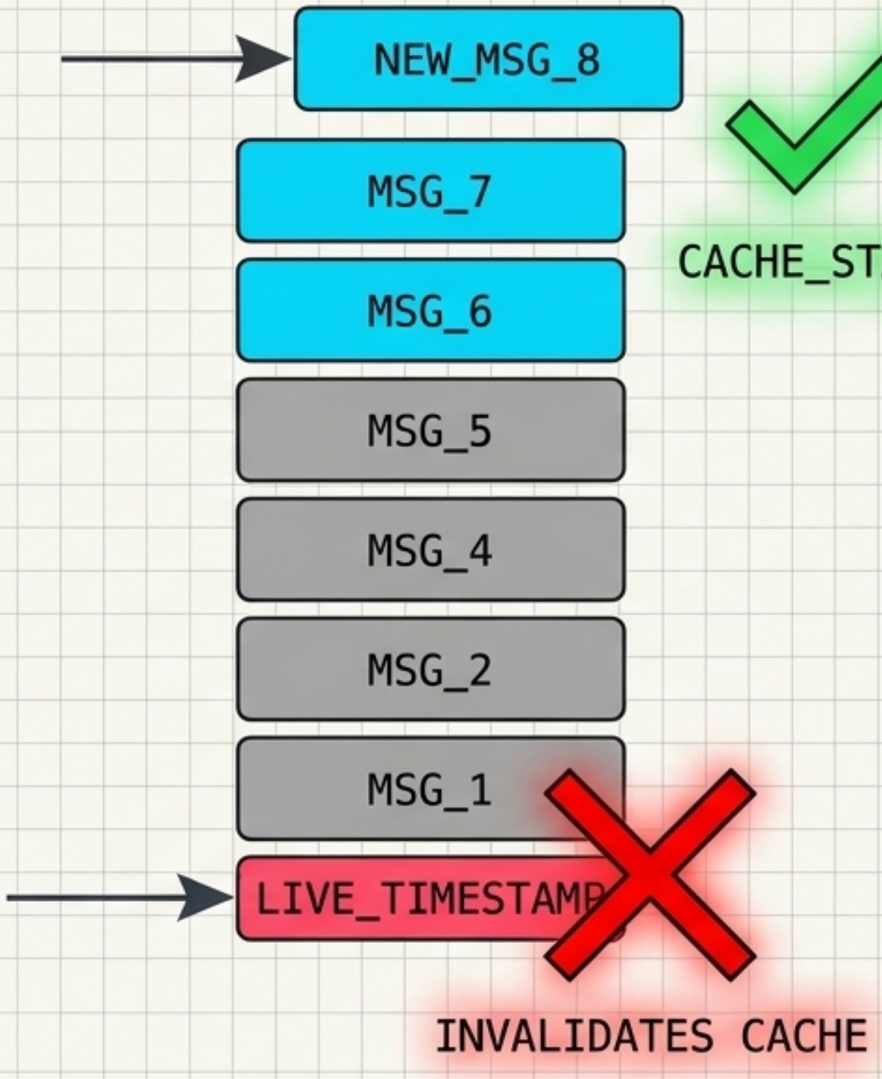
Code execution terminal

```
# The model does not get a dynamic tool schema.
# It gets a bash tool and writes this instead:
from sdk.tools.stripe_tools import list_subscriptions
subs = await list_subscriptions(status='active', limit=100)
```

Exposing 3,200+ integrations as python SDK functions keeps the prompt prefix perfectly stable forever. The model simply writes code to access them.

NotebookLM

Fix 2: The Append-Only Log



No Clocks in System Prompts

Injecting the current time at the top invalidates every cache on every call. Instead, the model infers time from timestamps strictly appended to new messages at the bottom.

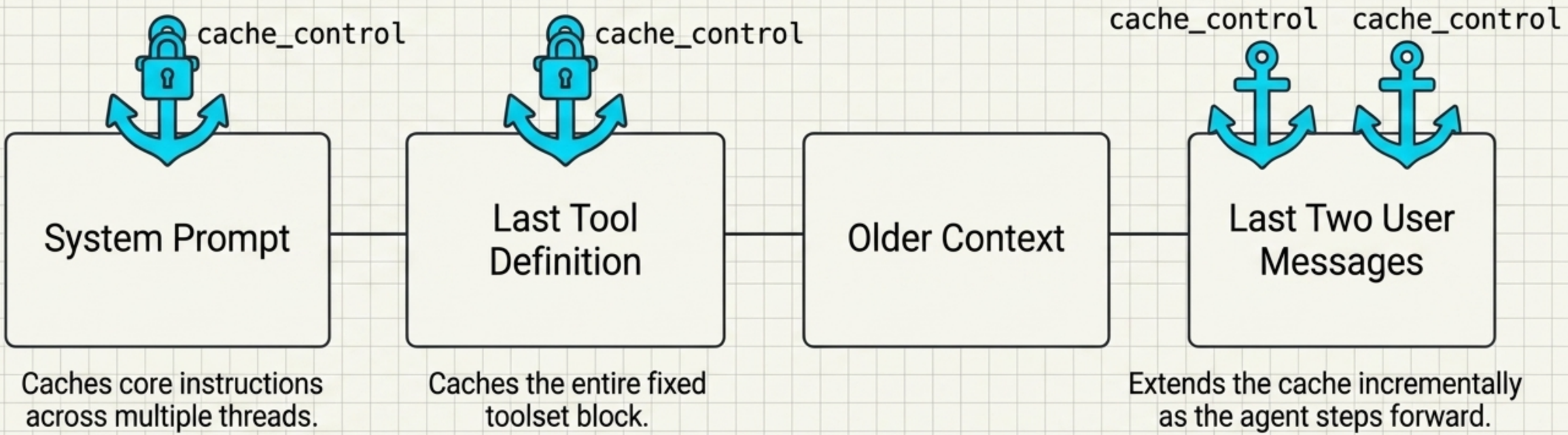
Mid-Thread Changes are Messages

If a user updates an instruction halfway through, you cannot edit the system prompt. The instruction update must be appended as a brand new system instruction message at the bottom of the log.

Cache safety must be a structural guarantee, not an engineering discipline. If nothing can be edited, nothing can accidentally torch a warm cache.

NotebookLM

Placing the Breakpoints

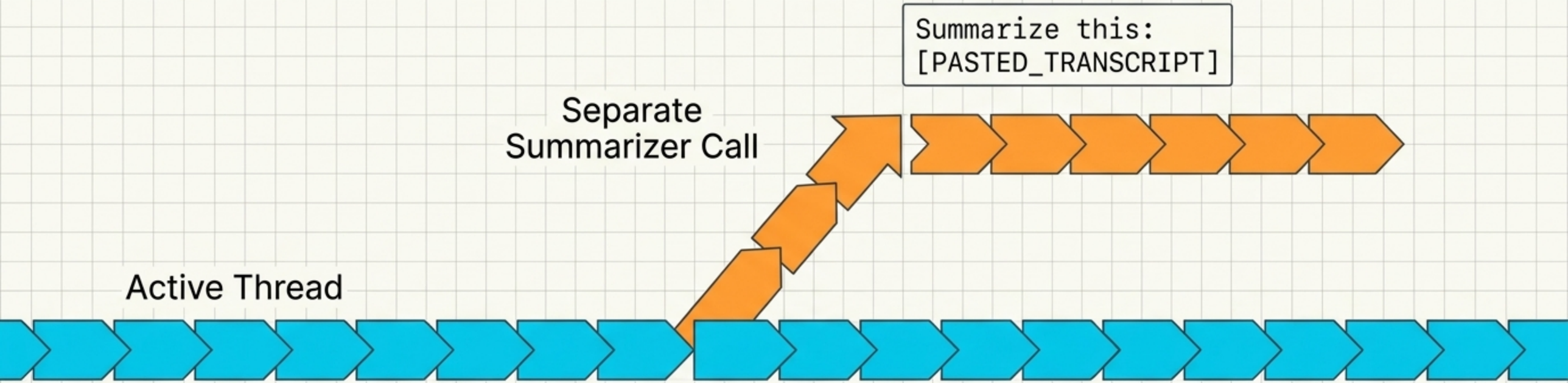



```
# Attaching breakpoints to the end of the conversation
user_messages = [i for i, msg in enumerate(messages) if msg['role'] == 'user']
indices_to_modify = user_messages[-2:]
item['cache_control'] = {'type': 'ephemeral'}
```

CONSTRAINT NOTE: Providers limit breakpoints. Anthropic allows **exactly 4 per request**, meaning strategic placement is mandatory.

NotebookLM

The Compaction Trap



The Trap

Long threads (200K+ tokens) must be compacted. But spinning up a separate summarizer call shares zero prefix with the live thread. The exact-match is instantly broken.

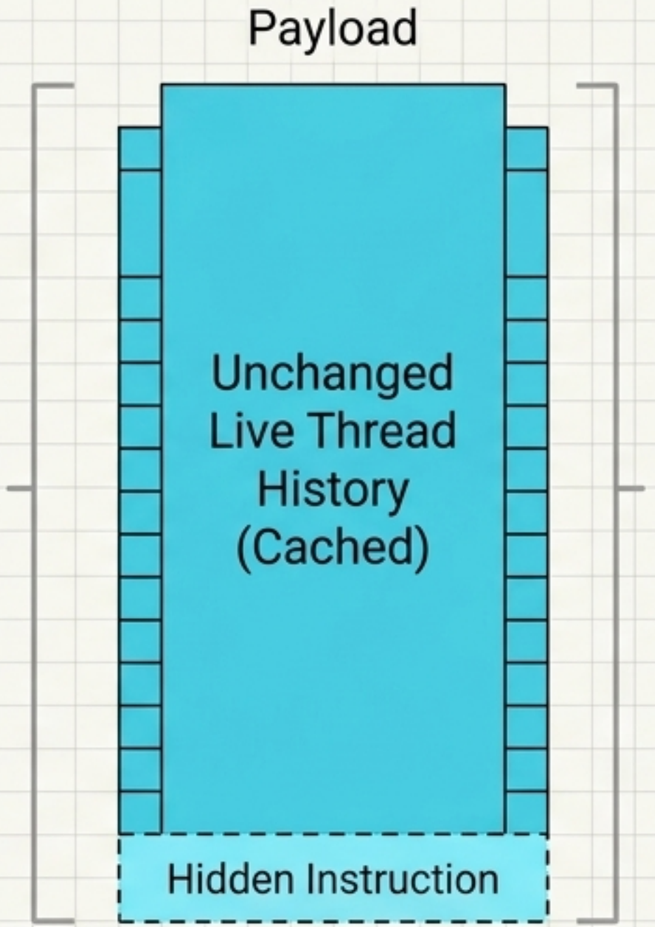
Economic Impact

You pay 100% full price ($0.30 per run) to re-read the massive 60k+ token history that you just spent hours carefully caching.

NotebookLM

In-Cache Compaction (The 0.1x Read Trick)

```
summary_messages.append({
  'role': 'user',
  'content': [{
    'type': 'text',
    'text': COMPACTION_PROMPT,
    SKIP_CACHE_CONTROL_KEY: True,
  }],
})
```

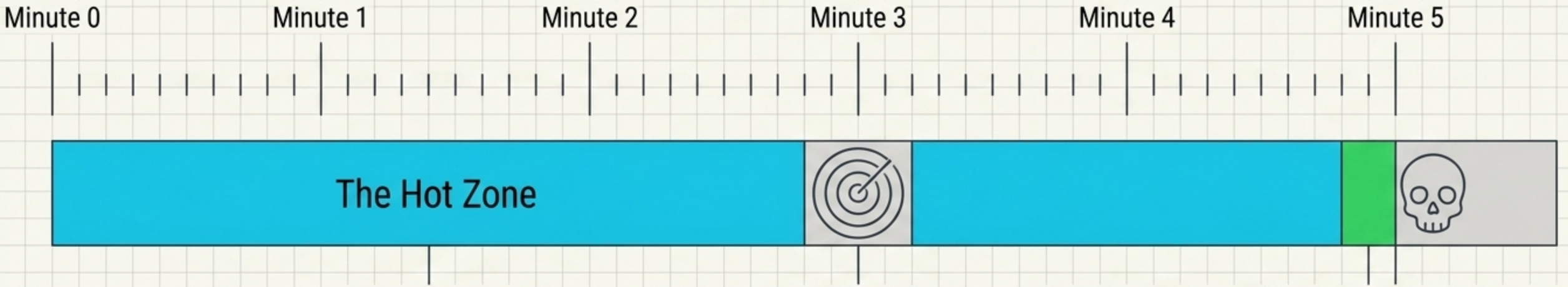


1. **Send the full history**:
   Transmit the entire live thread perfectly unchanged.
2. **Save the breakpoint**:
   Append the summary instruction but skip adding a cache breakpoint to preserve the 4-slot limit.
3. **Drop the tools**:
   Set tool_choice='none' to force a text summary without mutating the tool definitions prefix.

RESULT: Sending a full 60K token transcript this way costs $0.03 (cache read).
A trimmed separate call would cost $0.30 (cache miss).

NotebookLM

The Compaction Clock



The Hot Zone
Never compact. Let it ride the warm cache until **~50K** tokens.

The Signal
Delayed Summary Signal fires. If thread stays quiet, background worker activates.

The Maintenance Window
Idle threads compacted at **~16K** tokens. Ride the dying 0.1x cache right before it expires.

Cache Death
Prefix expires.

KEY CONCEPT: Compacting rewrites history and instantly invalidates the cache. Therefore, **never** amputate a hot thread. Wait until the cache is about to expire anyway, and use its dying breath to summarize cheaply, keeping the most recent 13K tokens verbatim.

NotebookLM

Provider Comparison Matrix

Dimension	Anthropic	OpenAI
Cache Trigger	Explicit (`cache_control` markers).	Automatic on prefixes >1K tokens.
Unit Economics	1.25x write penalty / 0.1x read.	No write penalty / 0.1x read.
Lifespan & Routing	Strict 5-minute sliding TTL.	Loose minutes-to-hour eviction, routed via `prompt_cache_key`.
Viktor Thread Strategy	Idle-clock compaction to beat the 5m TTL.	Less focus on cold-starts; high hot-thread threshold matters more.

TAKEAWAY: The agent architecture upstream is identical, but the thread engine adapts to wildly different downstream cache physics. Edge/Self-Hosted providers additionally require session-affinity headers.

NotebookLM

The Payoff: Unit Economics at Scale

~~$11.35~~ → **$2.07**

Context: 40-step agent thread processing 2.17M tokens on Claude Opus 4.8.	**Impact:** 81.8% raw API cost reduction.	**Latency:** Minutes of wall-clock latency eradicated by bypassing prefill compute.

TAKEAWAY: At the scale of thousands of threads a day, cache optimization is the difference between bleeding money on complex tasks and maintaining **highly profitable unit economics**.

NotebookLM

Synthesis: 5 Rules of Cache-Optimized Agents

	**1. Respect the Lifecycle:** Treat the provider's cache as a first-class physical component, not a billing checkbox.
	**2. Stable First:** Architect all conversation memory as a strict append-only log. No edits, no clocks.
	**3. Protect Position Zero:** Keep dynamic tool schemas out of the prompt. Expose integrations via SDKs.
	**4. In-Cache Maintenance:** Summarize inside the live thread's cache, not beside it in a separate full-price call.
	**5. Time Your Execution:** Never compact a hot thread. Perform expensive history rewrites the minute before a quiet cache dies.

NotebookLM